

# GATE DA Study Guide

---

## GATE DA Study Guide: Recursion · Complexity · Uninformed Search

---

### PART 1 — Recursion & Recurrence Writing

---

#### 1.1 Call stack & recursion depth

Every recursive call pushes a new **stack frame** (local variables, parameters, return address) onto the call stack. The **depth** of recursion = number of frames alive simultaneously = number of *pending* (not-yet-returned) calls.

**Key exam facts:** - Space complexity of a recursive function  $\approx O(\text{max recursion depth})$ , *not*  $O(\text{total number of calls})$ . - A function is **tail recursive** if the recursive call is the *last* operation (nothing pending after it returns). Compilers can optimize tail recursion into a loop ( $O(1)$  space in principle), but **C/Python/Java do NOT do this optimization by default** — GATE assumes no TCO unless stated. - Stack frames are popped in **LIFO** order — last call made, first to return.

**Tracing example:**

```
f(3) calls f(2) calls f(1) calls f(0) → returns
```

Stack depth reaches 4 ( $f(3), f(2), f(1), f(0)$ ) before any unwinding.

#### 1.2 Three classic recursion patterns

##### (a) Factorial — linear recursion

```
int fact(int n){
    if(n==0) return 1;
    return n*fact(n-1);
}
```

- Recurrence:  $T(n) = T(n-1) + O(1)$ ,  $T(0)=O(1)$
- Solving by unrolling:  $T(n) = T(n-1)+c = T(n-2)+2c = \dots = T(0)+nc \rightarrow T(n) = \Theta(n)$
- Max recursion depth =  $n+1$ . Space =  $O(n)$  (stack).

##### (b) Fibonacci — tree recursion (naive)

```
int fib(int n){
    if(n<=1) return n;
    return fib(n-1)+fib(n-2);
}
```

- Recurrence:  $T(n) = T(n-1) + T(n-2) + O(1)$
- This is NOT solvable by Master theorem (unequal subproblem sizes). Grows like Fibonacci numbers  $\rightarrow T(n) = \Theta(\varphi^n)$ ,  $\varphi \approx 1.618$  (exponential).
- Common GATE trap: total number of calls  $\approx 2 \cdot \text{fib}(n+1) - 1$ , i.e., exponential, even though depth is only  $O(n)$ .
- Max recursion depth =  $n$  (linear stack depth despite exponential total calls) — a favorite “spot the difference” MCQ.

## (c) GCD — Euclidean recursion

```
int gcd(int a, int b) {
    if (b == 0) return a;
    return gcd(b, a % b);
}
```

- This is tail recursion.
- Recurrence isn't simply  $T(n) = T(n-1) + O(1)$ ; depth is  $O(\log(\min(a, b)))$  — much faster than subtracting repeatedly, because  $a \% b$  shrinks arguments at least by half every two steps (Lamé's theorem, related to Fibonacci numbers giving the worst case).
- GATE angle: "What is the maximum number of recursive calls for  $\text{gcd}(a, b)$  where  $a, b \leq N$ ?" →  $O(\log N)$ .

### 1.3 Writing recurrences from code (general method)

Steps: 1. Identify the **base case** cost (usually  $O(1)$ ). 2. Identify **how many recursive calls** are made and on **what reduced input size**. 3. Add the **non-recursive (glue) work** done per call (loops, comparisons, merging).

$T(n) = (\text{number of subproblems}) \cdot T(\text{subproblem size}) + (\text{work outside recursive calls})$

Examples: | Code pattern | Recurrence | |—|—| | Single call,  $n-1$  |  $T(n) = T(n-1) + O(1)$  | | Single call,  $n-1$ , with an  $O(n)$  loop before/after |  $T(n) = T(n-1) + O(n)$  | | Two calls each on  $n/2$  (merge sort) |  $T(n) = 2T(n/2) + O(n)$  | | Two calls each on  $n-1$  (naive Fibonacci) |  $T(n) = 2T(n-1) + O(1)$  (*upper bound style*) | | One call on  $n/2$  + one call on  $n/2$  with extra  $O(1)$  (binary search) |  $T(n) = T(n/2) + O(1)$  | | Call on  $2n+1$  and  $2n$  (as seen in PYQ style) |  $T(N) = T(N/2) + O(\log N)$  (*after substitution*) |

## PART 2 — Complexity Analysis (Big-O, $\Theta$ , $\Omega$ )

### 2.1 Formal definitions (memorize exactly — GATE tests the definitions directly)

Let  $f(n)$ ,  $g(n)$  be functions from positive integers to positive reals.

- **$O(g(n))$**  (upper bound):  $f(n) = O(g(n))$  if  $\exists$  positive constants  $c, n_0$  such that  $0 \leq f(n) \leq c \cdot g(n)$  for all  $n \geq n_0$
- **$\Omega(g(n))$**  (lower bound):  $f(n) = \Omega(g(n))$  if  $\exists c, n_0$  such that  $0 \leq c \cdot g(n) \leq f(n)$  for all  $n \geq n_0$
- **$\Theta(g(n))$**  (tight bound):  $f(n) = \Theta(g(n))$  if  $\exists c_1, c_2, n_0$  such that  $0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$  for all  $n \geq n_0$   $\Leftrightarrow f(n) = O(g(n))$  AND  $f(n) = \Omega(g(n))$

**Common confusions GATE exploits:** -  $O$  is like " $\leq$ ",  $\Omega$  is like " $\geq$ ",  $\Theta$  is like " $=$ " — but these are about *growth rate*, not exact value. -  $f(n) = O(g(n))$  does **not** imply  $g(n) = O(f(n))$ . - Little-o ( $o$ ) means strict  $<$  for all large-enough  $n$  as constants  $\rightarrow \infty$  ( $f$  grows *strictly* slower); little-omega ( $\omega$ ) is the strict lower analog. GATE occasionally asks to distinguish  $o/\omega$  from  $O/\Omega$ . - A function can have multiple valid  $O$  bounds (e.g.,  $n = O(n^2)$  is TRUE, just not tight) but only one  $\Theta$  (up to constants).

### 2.2 Nested loop complexity — quick recipes

| Loop structure                          | Complexity                                  |
|-----------------------------------------|---------------------------------------------|
| for i=1..n: for j=1..n: (independent)   | $\Theta(n^2)$                               |
| for i=1..n: for j=1..i: (triangular)    | $\Theta(n^2)$ — sum = $n(n+1)/2$            |
| for i=1..n: for j=1; j<n; j*=2:         | $\Theta(n \log n)$ — inner loop is $\log n$ |
| for i=1; i<n; i*=2: for j=1..n:         | $\Theta(n \log n)$                          |
| for i=1; i<n; i*=2: for j=1; j<i; j*=2: | $\Theta(\log^2 n)$                          |

|                                           |                  |
|-------------------------------------------|------------------|
| for i=1..n: for j=1..n: for k=1..n:       | $\Theta(n^3)$    |
| While loop halving n each time, O(1) work | $\Theta(\log n)$ |

**Trick to remember:** if the inner-loop bound depends on the outer variable additively (j from 1 to i), you still typically get  $\Theta(n^2)$  (sum of an arithmetic series). If the loop variable is *multiplied/divided* each iteration, you get a **log** factor.

### 2.3 Reading Big-O of code snippets — confident 5-step check

1. Any single loop from 1 to n with O(1) body  $\rightarrow O(n)$ .
2. Any loop that multiplies/divides the counter  $\rightarrow O(\log n)$ .
3. Nested loops  $\rightarrow$  **multiply** the individual complexities (only exactly if loops are independent; add carefully if inner bound depends on outer).
4. Sequential (non-nested) blocks  $\rightarrow$  **add** complexities, then keep only the dominant term.
5. A recursive call  $\rightarrow$  write its recurrence and solve (Part 3).

## PART 3 — Solving Recurrences

### 3.1 Substitution method (guess + prove by induction)

**Method:** (1) Guess the closed form (often from unrolling/iteration). (2) Assume it holds for smaller n. (3) Prove it holds for n by substitution into the recurrence. (4) Adjust constants if needed.

**Solved Problem 1:**  $T(n) = T(n-1) + n$ ,  $T(1) = 1$  - Unroll:  $T(n) = T(n-1) + n = T(n-2) + (n-1) + n = \dots = T(1) + 2 + 3 + \dots + n = 1 + (n(n+1)/2 - 1)$  -  **$T(n) = \Theta(n^2)$**  - Verify by substitution: guess  $T(n) \leq cn^2$ .  $T(n) = T(n-1) + n \leq c(n-1)^2 + n = cn^2 - 2cn + c + n$ . Need  $\leq cn^2$ , i.e. need  $-2cn + c + n \leq 0$ , true for  $c \geq 1$  and  $n \geq 1$ . ✓ upper bound holds.

**Solved Problem 2:**  $T(n) = 2T(n/2) + n$ ,  $T(1) = 1$  - Guess  $T(n) = O(n \log n)$ . Assume  $T(k) \leq ck \log k$  for  $k < n$ . -  $T(n) = 2T(n/2) + n \leq 2 \cdot c(n/2) \log(n/2) + n = cn \log(n/2) + n = cn \log n - cn + n \leq cn \log n$  provided  $c \geq 1$ . ✓  **$T(n) = \Theta(n \log n)$**

**Solved Problem 3:**  $T(n) = T(n/2) + 1$ ,  $T(1) = 1$  (binary search style) - Let  $n = 2^k \rightarrow T(2^k) = T(2^{k-1}) + 1 = \dots = T(1) + k = 1 + \log_2 n$  -  **$T(n) = \Theta(\log n)$**

**Solved Problem 4:**  $T(n) = T(\sqrt{n}) + 1$  (a favorite tricky GATE substitution) - Let  $n = 2^m$ , so  $\sqrt{n} = 2^{m/2}$ . Define  $S(m) = T(2^m)$ . -  $S(m) = S(m/2) + 1 \rightarrow$  by problem 3's pattern,  $S(m) = \Theta(\log m)$  - Since  $m = \log_2 n$ ,  **$T(n) = \Theta(\log \log n)$**

### 3.2 Master theorem

For  $T(n) = a \cdot T(n/b) + f(n)$ ,  $a \geq 1$ ,  $b > 1$ , compare  $f(n)$  with  $n^{\log_b a}$ :

| Case | Condition                                                                                                                | Result                                                        |
|------|--------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------|
| 1    | $f(n) = O(n^{\log_b a - \epsilon})$ for some $\epsilon > 0$                                                              | $T(n) = \Theta(n^{\log_b a})$                                 |
| 2    | $f(n) = \Theta(n^{\log_b a})$                                                                                            | $T(n) = \Theta(n^{\log_b a} \cdot \log n)$                    |
| 3    | $f(n) = \Omega(n^{\log_b a + \epsilon})$ <b>and</b> regularity: $a \cdot f(n/b) \leq c \cdot f(n)$ for $c < 1$ , large n | $T(n) = \Theta(f(n))$                                         |
| —    | none of the above (polynomial gap missing, or f smaller but not polynomially)                                            | Master theorem doesn't apply — use Akra-Bazzi or substitution |

## 5 Solved Problems:

1.  $T(n) = 2T(n/2) + n$   $a=2, b=2 \rightarrow n^{\log_2 2} = n^1$ .  $f(n) = n = \Theta(n^1) \rightarrow$  **Case 2**  $\rightarrow T(n) = \Theta(n \log n)$  (this is merge sort).
2.  $T(n) = 4T(n/2) + n$   $a=4, b=2 \rightarrow n^{\log_2 4} = n^2$ .  $f(n) = n = O(n^{2-\epsilon}) \rightarrow$  **Case 1**  $\rightarrow T(n) = \Theta(n^2)$ .
3.  $T(n) = T(n/2) + n$   $a=1, b=2 \rightarrow n^{\log_2 1} = n^0 = 1$ .  $f(n) = n = \Omega(n^{0+1})$  and regularity holds  $\rightarrow$  **Case 3**  $\rightarrow T(n) = \Theta(n)$ .
4.  $T(n) = 2T(n/2) + n^2$   $a=2, b=2 \rightarrow n^1$ .  $f(n) = n^2 = \Omega(n^{1+1})$ ; check regularity:  $2 \cdot (n/2)^2 = n^2/2 \leq c \cdot n^2$  for  $c=1/2 < 1$   $\checkmark \rightarrow$  **Case 3**  $\rightarrow T(n) = \Theta(n^2)$ .
5.  $T(n) = 3T(n/2) + n$   $a=3, b=2 \rightarrow n^{\log_2 3} \approx n^{1.585}$ .  $f(n) = n = O(n^{1.585-\epsilon}) \rightarrow$  **Case 1**  $\rightarrow T(n) = \Theta(n^{\log_2 3})$ .

**Where Master theorem FAILS (know these for MCQs):** -  $T(n) = 2T(n/2) + n \log n \rightarrow f(n) = n \log n$  is *not* polynomially larger than  $n^{\log_2 2} = n$  (only by a log factor)  $\rightarrow$  gap between Case 2 and 3, Master theorem's polynomial-gap requirement fails. (Solved instead via generalized/Akra-Bazzi:  $\Theta(n \log^2 n)$ .) -  $T(n) = T(n/2) + 2^n \rightarrow$  not of form  $aT(n/b) + f(n)$  growth mismatch is extreme; here  $f$  dominates so hugely,  $T(n) = \Theta(2^n)$ , but always double check with the regularity condition, not just growth comparison. - Unequal-sized subproblems, e.g.  $T(n) = T(n/3) + T(2n/3) + n \rightarrow$  Master theorem does **not apply** (needs same  $n/b$  for both terms); solved via recursion tree (answer:  $\Theta(n \log n)$ ).

## PART 4 — Uninformed Search

### 4.1 BFS (Breadth-First Search)

- **Data structure:** FIFO Queue.
- **Strategy:** Explore all nodes at depth  $d$  before any node at depth  $d+1$  (level-order).
- **Completeness:** Yes (if branching factor  $b$  is finite).
- **Optimality:** Yes, **if all step costs are equal** (e.g., unweighted graph). Not optimal for general weighted costs.
- **Time complexity:**  $O(b^d)$ , where  $b$  = branching factor,  $d$  = depth of shallowest goal.
- **Space complexity:**  $O(b^d)$  — BFS must store the entire frontier (exponential space is BFS's main weakness).
- Used for: shortest path in unweighted graphs, level-order traversal, finding connected components.

### 4.2 DFS (Depth-First Search)

- **Data structure:** LIFO Stack (or recursion).
- **Strategy:** Go as deep as possible along one branch before backtracking.
- **Completeness:** **No** in infinite/cyclic state spaces without depth limit or visited-check (can loop forever); Yes in finite graphs with visited tracking.
- **Optimality:** **No** — may find a longer path first.
- **Time complexity:**  $O(b^m)$ ,  $m$  = maximum depth of search tree (can be much worse than BFS's  $d$  if  $m \gg d$ ).
- **Space complexity:**  $O(b \cdot m)$  — only needs to store nodes on current path plus siblings, **much better than BFS**.
- Trade-off to remember: **DFS wins on space, BFS wins on optimality/shallow-goal speed**.

### 4.3 UCS (Uniform Cost Search)

- **Data structure:** Priority queue ordered by cumulative path cost  $g(n)$  (min-cost first).
- **Strategy:** Always expand the node with lowest cumulative cost so far — generalizes BFS to weighted graphs (BFS = UCS when all edge costs equal).

- **Completeness:** Yes, if step costs  $\geq$  some small positive  $\epsilon$ .
- **Optimality: Yes** — always finds the least-cost path (this is UCS's main advantage over BFS in weighted graphs).
- **Time/Space complexity:**  $O(b^{(1+\lceil C/\epsilon \rceil)})$ , where  $C$  = cost of optimal solution,  $\epsilon$  = minimum edge cost — can be far worse than  $O(b^d)$  if there are many cheap edges before reaching the goal.
- **Key implementation detail:** a node may be re-inserted with a better cost if a cheaper path to it is found before it's finalized (like Dijkstra's algorithm — in fact **UCS  $\equiv$  Dijkstra's algorithm** run until goal is popped).

#### 4.4 Quick comparison table (high-yield for MCQs)

| Property       | BFS               | DFS                       | UCS                                     |
|----------------|-------------------|---------------------------|-----------------------------------------|
| Data structure | Queue (FIFO)      | Stack (LIFO)              | Priority Queue                          |
| Complete?      | Yes (finite $b$ ) | No (infinite depth/loops) | Yes ( $\epsilon$ -cost)                 |
| Optimal?       | Yes (unit cost)   | No                        | Yes (general cost)                      |
| Time           | $O(b^d)$          | $O(b^m)$                  | $O(b^{(1+\lceil C^*/\epsilon \rceil)})$ |
| Space          | $O(b^d)$          | $O(b \cdot m)$            | $O(b^{(1+\lceil C^*/\epsilon \rceil)})$ |

## PART 5 — GATE-style Previous-Year-Pattern Questions (PYQs)

These reproduce the well-known style/content of standard GATE Algorithms questions on these exact topics — solve them like real GATE questions.

**Q1.** Consider the recurrence  $T(n) = 2T(n/2) + n$ ,  $T(1) = 1$ . What is  $T(n)$ ? (a)  $\Theta(n)$  (b)  $\Theta(n \log n)$  (c)  $\Theta(n^2)$  (d)  $\Theta(\log n)$  Answer: (b), by Master theorem Case 2.

**Q2.** The recurrence relation for the number of calls made by the naive recursive Fibonacci function  $\text{fib}(n)$  is best described as: (a)  $T(n) = T(n-1) + O(1)$  (b)  $T(n) = T(n-1) + T(n-2) + O(1)$  (c)  $T(n) = 2T(n/2) + O(1)$  (d)  $T(n) = T(n-1)^2$  Answer: (b).

**Q3.** Which of the following gives the maximum recursion depth of Euclid's  $\text{gcd}(a,b)$  algorithm when  $a, b \leq N$ ? (a)  $O(N)$  (b)  $O(\sqrt{N})$  (c)  $O(\log N)$  (d)  $O(N \log N)$  Answer: (c).

**Q4.** For  $T(n) = 3T(n/4) + n \log n$ , applying the Master theorem: (a) Case 1 applies,  $T(n) = \Theta(n^{(\log_4 3)})$  (b) Case 2 applies,  $T(n) = \Theta(n \log^2 n)$  (c) Case 3 applies,  $T(n) = \Theta(n \log n)$  (d) Master theorem does not apply Answer: (c). Since  $\log_4 3 \approx 0.79$ ,  $f(n) = n \log n = \Omega(n^{(0.79+\epsilon)})$ , and regularity condition holds  $\rightarrow$  Case 3.

**Q5.** In BFS on a graph with branching factor  $b$ , if the shallowest goal is at depth  $d$ , the worst-case space complexity is: (a)  $O(bd)$  (b)  $O(b^d)$  (c)  $O(d^b)$  (d)  $O(b \cdot m)$  Answer: (b).

**Q6.** Which uninformed search strategy is guaranteed to find the optimal-cost path when edge costs are non-negative and unequal? (a) BFS (b) DFS (c) UCS (d) Iterative deepening DFS (no cost awareness) Answer: (c).

**Q7.** If  $f(n) = O(g(n))$  and  $g(n) = O(h(n))$ , then which of the following must be TRUE? (a)  $f(n) = \Theta(h(n))$  (b)  $f(n) = O(h(n))$  (c)  $h(n) = O(f(n))$  (d)  $f(n) = \Omega(h(n))$  Answer: (b) — transitivity of  $O$ .

**Q8.** Consider: `for(i=1; i<n; i=i*2) for(j=0; j<i; j++) sum++;` What is the time complexity? (a)  $\Theta(n)$  (b)  $\Theta(n \log n)$  (c)  $\Theta(\log n)$  (d)  $\Theta(n^2)$  Answer: (a). Inner loop runs  $i$  times for  $i=1, 2, 4, \dots, n/2$  — sum is a geometric series  $\approx n-1 = \Theta(n)$ . (Classic GATE trap — looks like it should be  $n \log n$  but the geometric sum collapses to  $\Theta(n)$ .)

**Q9.**  $T(n) = T(n/3) + T(2n/3) + n$ . What is  $T(n)$ ? (Recursion-tree / cannot use Master theorem directly)

(a)  $\Theta(n)$  (b)  $\Theta(n \log n)$  (c)  $\Theta(n^2)$  (d)  $\Theta(n \log^2 n)$  Answer: (b). Every level of the (unbalanced) recursion tree sums to  $n$  in total work; tree height is  $\Theta(\log n) \rightarrow \Theta(n \log n)$ .

**Q10.** DFS is preferred over BFS mainly because: (a) DFS always finds shorter paths (b) DFS has lower space complexity  $O(b \cdot m)$  vs BFS's  $O(b^d)$  (c) DFS is always complete (d) DFS never revisits nodes Answer: (b).

---

## PART 6 — Extra Practice Set (self-check, answers below each)

---

**P1.** Write the recurrence for the number of recursive calls made by `power(x, n)` defined as `power(x, n) = power(x, n/2)²` if  $n$  even, `x * power(x, n-1)` if  $n$  odd. What's the time complexity? Hint:  $T(n) = T(n/2) + O(1)$  dominates  $\rightarrow \Theta(\log n)$ .

**P2.** What is the maximum stack depth for `fib(n)` implemented as plain double recursion (not memoized)? Answer:  $O(n)$  — despite exponential total calls, the deepest single path is  $n$  frames.

**P3.** Solve by substitution:  $T(n) = 2T(n-1) + 1$ ,  $T(0)=1$ . Answer:  $T(n) = 2^{n+1} - 1 = \Theta(2^n)$ .

**P4.** Solve by substitution:  $T(n) = T(n-1) + \log n$ ,  $T(1)=0$ . Answer:  $T(n) = \Theta(n \log n)$  (sum of  $\log 2 + \log 3 + \dots + \log n = \log(n!) = \Theta(n \log n)$ ).

**P5.** Apply Master theorem:  $T(n) = 8T(n/2) + n^2$ .  $a=8, b=2 \rightarrow n^3$ ;  $f(n)=n^2 = O(n^{(3-\epsilon)}) \rightarrow$  Case 1  $\rightarrow \Theta(n^3)$ .

**P6.** Apply Master theorem:  $T(n) = T(n/2) + \log n$ .  $a=1, b=2 \rightarrow n^0=1$ ;  $f(n)=\log n = \Omega(n^{(0+\epsilon)})$ ? No —  $\log n$  is NOT polynomially larger than a constant for any  $\epsilon > 0$ . This is the tricky “in-between” case — Master theorem (basic 3-case form) doesn't strictly cover it, but by direct summation  $T(n) = \Theta(\log^2 n)$ .

**P7.** Nested loop: `for(i=1; i<=n; i++) for(j=1; j<=n; j*=2) x++;` — Complexity? Answer:  $\Theta(n \log n)$ .

**P8.** A graph has branching factor 4. BFS finds a goal at depth 3, while DFS (unlucky ordering) explores to max depth 10 before finding a (worse) solution. Compare worst-case node expansions. BFS:  $O(4^3)=64$ . DFS:  $O(4^{10}) \approx 10^6$  — illustrates why unbounded DFS can be far costlier despite lower space per branch.

**P9.** In UCS, when is UCS identical in behavior to BFS? Answer: When all edge costs are equal (e.g., all = 1) — UCS then expands nodes in the same order as BFS.

**P10.** True/False:  $\Theta(n)$  implies both  $O(n)$  and  $\Omega(n)$ . Answer: True — by definition,  $\Theta$  is the intersection of  $O$  and  $\Omega$ .

---

## Quick Revision Checklist

---

- ☐ Can trace call-stack depth vs. total calls separately (Fibonacci trap)
- ☐ Can write a recurrence directly from code in under 30 seconds
- ☐ Know  $O/\Theta/\Omega$  definitions word-for-word, not just intuitively
- ☐ Can identify nested-loop complexity without writing out full summation, using the “additive  $\rightarrow n^2$ , multiplicative  $\rightarrow \log$ ” rule
- ☐ Comfortable with substitution method: guess  $\rightarrow$  assume  $\rightarrow$  prove
- ☐ Master theorem: know all 3 cases + when it does NOT apply (equal-size check, polynomial-gap check, regularity condition)
- ☐ BFS/DFS/UCS: data structure, completeness, optimality, time & space — cold recall of the comparison table